Universidad Católica del Uruguay

Facultad de Ingeniería y Tecnologías

Ingeniería en Informática

Informe de Proyecto

Algoritmos y Estructuras de Datos

Proyecto Integrador 2: Sistema de Telecomunicaciones

Equipo 3

Santiago Acosta

Milagros García

Federico Martínez

Bruno Pereda

Alexis Giménez

28 de octubre de 2026

Índice

1. Introducción 1
2. Descripción general de la solución 1
2.1. Biblioteca de grafos 1
2.2. Modelado de la red de telecomunicaciones 3
2.3. Operaciones principales del sistema 3
3. Representación de los grafos 4
4. Principales decisiones de diseño 5
5. Análisis de complejidad 7
6. Decisiones tomadas en el Desafío 2 9
7. Optimización realizada en el Desafío 3 10
8. Resultados de los experimentos realizados 12
9. Declaración de uso de herramientas de IA 13
10. Conclusión 14

1. Introducción

El presente informe corresponde al Proyecto Integrador 2 de Algoritmos y Estructuras de Datos, en el que se pasa de las estructuras lineales y jerárquicas de los hitos anteriores a los grafos. El trabajo se organizó en los tres desafíos que plantea la letra. En el primero se construyó una biblioteca propia de grafos dirigidos y no dirigidos junto con sus algoritmos principales. En el segundo esa biblioteca se usó para modelar la red de datos de un proveedor de telecomunicaciones, que conecta oficinas, puntos de acceso y data centers mediante enlaces de fibra, cobre y radioenlace. En el tercero se identificó una operación del sistema que se vuelve un cuello de botella a medida que la red crece, se propuso una alternativa y se comparó experimentalmente con la versión original.

A diferencia de los hitos anteriores, buena parte del trabajo no estuvo en implementar los algoritmos sino en decidir qué representa cada parte del grafo. Una misma red se puede mirar como el conjunto de caminos por los que viajan los datos, como un plano de cableado o como algo que puede romperse, y cada una de esas miradas pide un grafo distinto: dirigido o no, con un peso o con otro. Muchas de las decisiones que se describen en este informe salen de esa pregunta.

A lo largo del informe se describen la biblioteca, el modelo de la red y las operaciones que ofrece el sistema. Se justifican tanto la representación elegida como las principales decisiones de diseño, se analiza el costo de las operaciones en función de la cantidad de nodos y de enlaces y, se presentan la optimización del Desafío 3 y los resultados del experimento realizado. Por último, se detalla el uso que se hizo de las herramientas de IA y se dan las conclusiones del equipo.

2. Descripción general de la solución

2.1. Biblioteca de grafos

El Desafío 1 consistió en implementar las interfaces del proyecto base de la unidad, la implementación está en el paquete ucu.edu.aed.tda.grafo, separada del sistema de telecomunicaciones, de modo que la biblioteca no conoce nada del dominio en el que se la usa.

DirectedGraph y UndirectedGraph guardan el grafo como listas de adyacencia: un LinkedHashMap que asocia cada vértice con la lista de aristas que salen de él. En el grafo no dirigido cada arista se guarda dos veces, una en la lista de cada extremo y con ese extremo como origen, porque la interfaz exige que adyacencias(v) devuelva aristas cuyo source() sea v. Para que el listado de aristas no las repita, UndirectedEdge define un equals simétrico, en el que (a, b) y (b, a) son la misma arista. El grafo dirigido acepta lazos, que en un dirigido son un ciclo de largo uno, mientras que el no dirigido los rechaza junto con las aristas repetidas, ya que en el teórico los ciclos de un grafo no dirigido tienen largo tres o más.

Sobre el grafo dirigido se implementaron Dijkstra, Floyd, Warshall, la excentricidad y el centro, todos los caminos entre dos vértices, las búsquedas en profundidad y en amplitud y la clasificación topológica. Dijkstra usa una cola de prioridad en lugar del arreglo D y el conjunto S del teórico: cada vez que mejora el costo de un vértice se encola una entrada nueva, y las entradas desactualizadas se descartan al salir de la cola. Floyd mantiene la matriz de costos A y la de intermedios P, donde -1 indica que el camino es el arco directo a diferencia de en el teórico donde se usa 0 ya que los vértices se numeran desde 1. La excentricidad de un vértice es el máximo de su columna en la matriz de Floyd, es decir, la mayor distancia desde los demás vértices hacia él, al igual que lo planteado en el teórico. La clasificación topológica recorre en profundidad siguiendo los predecesores, lo que equivale a invertir los arcos, y agrega cada vértice a la salida al terminar su recursión. Todos los caminos se obtienen con una búsqueda en profundidad con vuelta atrás, que desmarca cada vértice al volver de él para poder usarlo en otro camino. Quedó incluido también el cálculo del camino crítico, que no forma parte de la interfaz en si, pero venía del práctico anterior.

Sobre el grafo no dirigido se implementaron Prim, Kruskal, la búsqueda en amplitud y los puntos de articulación. Prim sigue la versión del teórico que usa buscarMin, que en la interfaz es searchMinEdge; en lugar de probar todos los pares de vértices, recorre las aristas de cada vértice ya incorporado al árbol y se queda con la más barata que lleva afuera de él. Si el grafo no es conexo, en algún momento buscarMin no encuentra ninguna arista y el resultado es el árbol de la componente del origen.

Kruskal se implementó con componentes: las aristas se ordenan por costo y se agrega cada una que une dos componentes distintas, que pasan a ser una sola. Se corta al llegar a $V - 1$ aristas o al quedarse sin aristas, así que en un grafo no conexo devuelve un bosque con un árbol de costo mínimo por componente. Este corte surgió de una diferencia con la diapositiva de la versión sin componentes, que repite hasta tener $\#A - 1$ aristas: la cantidad correcta es $\#V - 1$, y sin el corte por falta de aristas el algoritmo no termina en un grafo no conexo.

Los puntos de articulación se calculan con una búsqueda en profundidad que numera los vértices en orden previo (numBp) y calcula bajo al volver de la recursión, dado como el mínimo entre: el número del propio vértice, el de los vértices alcanzados por aristas de retroceso y el bajo de sus hijos. La raíz del recorrido es punto de articulación si tiene dos o más hijos, y cualquier otro vértice v lo es si tiene un hijo w con bajo[w] >= numBp[v]. Se arranca una búsqueda por cada componente, así que el método funciona también en grafos no conexos.

La biblioteca se probó con 202 casos de prueba en JUnit. Además de los casos borde que pide la letra (grafo vacío, un único vértice, grafos conexos y no conexos, con y sin ciclos, vértices inexistentes, pesos iguales y caminos alternativos de igual costo), se incluyeron los ejemplos de las diapositivas con su resultado exacto, como la matriz completa de Floyd o el árbol de costo 15 del grafo de seis vértices, y comparaciones cruzadas entre algoritmos: Floyd contra Dijkstra corrido desde cada vértice, y los puntos de articulación contra su definición en 50 grafos generados al azar con semilla fija, sacando cada vértice y contando si aumentan las componentes conexas. Para comprobar que las pruebas detectaban errores reales, se introdujeron errores a propósito en copias del código, por ejemplo cambiar >= por > en la condición de articulación, y se verificó que los tests fallaran como sería esperado.

2.2. Modelado de la red de telecomunicaciones

Comenzando con el Nodo, este tiene un identificador, un tipo, una capacidad y una cantidad de usuarios. Los tipos son ROUTER, SWITCH, SERVIDOR, PUNTO_ACCESO, DATA_CENTER y OFICINA, y solo los puntos de acceso y las oficinas atienden usuarios; en ellos la capacidad es la cantidad máxima de usuarios que pueden atender, y en el resto es un dato informativo del equipo. Un Enlace une dos nodos y tiene un tipo entre FIBRA, COBRE o RADIOENLACE, una latencia en cada sentido, un ancho de banda y un costo de instalación. Cada tipo de enlace define el rango de ancho de banda que admite, y un enlace se rechaza si su valor queda fuera de ese rango. Las dos clases validan sus datos en el constructor (latencias positivas, costo no negativo, usuarios solo en nodos de acceso y nunca más que la capacidad), así que no puede existir un nodo o un enlace inconsistente.

RedTelecom guarda la red completa una sola vez: los nodos en un mapa por identificador y los enlaces en un mapa cuya clave es el par de extremos sin orden. Para lograrlo, la clave se arma con los dos identificadores ordenados, de modo que A-B y B-A dan la misma clave y encontrar el enlace entre dos nodos cuesta O(1) en cualquier orden. A partir de esa información se arman tres grafos, uno para cada necesidad, y en todos ellos los vértices son los identificadores de los nodos. Los atributos de los equipos no se guardan en el grafo, cuando un servicio necesita los usuarios de un nodo o el ancho de banda de un enlace, los busca en RedTelecom.

El grafo de ruteo es dirigido y su peso es la latencia. Cada enlace a-b produce dos arcos, a → b con la latencia de a hacia b y b → a con la de b hacia a, porque la latencia de un enlace no tiene por qué ser la misma en los dos sentidos. Este grafo tiene una variante que incluye solo los enlaces con al menos cierto ancho de banda y otra que deja afuera los nodos y enlaces caídos. Es el que usan el ruteo y la distribución de contenido.

El grafo de cableado es no dirigido y su peso es el costo de instalación. Instalar un enlace cuesta lo mismo sin importar desde qué extremo se lo mire, y el árbol abarcador de costo mínimo es un problema de grafos no dirigidos.

El grafo de resiliencia es no dirigido y sin peso (todas las aristas valen 1). Cuando un enlace se corta deja de funcionar en los dos sentidos, y los puntos de articulación también se definen sobre grafos no dirigidos. Las fallas se representan pasando los nodos y enlaces caídos como parámetro: el grafo se arma sin ellos y la red original nunca se modifica.

Los tres grafos se arman de nuevo en cada llamada. Esto tiene un costo, que se analiza en la sección 5, pero garantiza que ningún servicio pueda modificar la red por accidente al trabajar sobre el grafo que recibió.

2.3. Operaciones principales del sistema

Las operaciones se agrupan en servicios del paquete ucu.edu.aed.telecom.servicio, y cada uno devuelve sus resultados en clases propias del paquete ucu.edu.aed.telecom.resultado.

ServicioRuteo resuelve por dónde deben viajar los datos. rutaMenorLatencia aplica Dijkstra sobre el grafo de ruteo y devuelve los nodos del camino, la latencia total y el cuello de botella,

que es el menor ancho de banda entre los enlaces de la ruta. rutaConAnchoBandaMinimo hace lo mismo sobre el grafo que tiene solo los enlaces suficientemente anchos, y rutaAlternativa sobre el grafo sin los elementos caídos. tablaDeRuteo aplica Floyd y devuelve una TablaRuteo que permite consultar, para cualquier par de nodos, la latencia mínima, el camino completo y el siguiente salto, que es lo que guarda un router en su tabla. hayCamino usa Warshall, y diametro busca en la matriz de Floyd el par de nodos con la mayor latencia mínima.

ServicioCableado resuelve la conexión al menor costo. cableadoMinimo aplica Kruskal sobre el grafo de cableado y devuelve los enlaces elegidos con su costo total, indicando si se pudo conectar a todos los nodos o si el resultado es un bosque.

ServicioResiliencia responde qué pasa cuando algo se rompe. nodosCriticos devuelve los puntos de articulación del grafo de resiliencia, enlacesCriticos devuelve los puentes, e incomunicados devuelve los nodos que siguen funcionando pero ya no llegan a ningún data center. Sobre estas operaciones se construyen impactoFallaNodo e impactoFallaEnlace, que informan qué nodos quedan incomunicados por una falla puntual y a cuántos usuarios afecta, y rankingCriticosSimple, que ordena los nodos según cuántos usuarios se quedarían sin servicio si cayeran.

ServicioCDN resuelve la distribución de contenido. asignarCDN aplica Floyd y le asigna a cada nodo el data center que lo atiende con menor latencia. recomendarDCSimple y su versión optimizada recomendarDC proponen dónde instalar un data center nuevo, centroParaDC calcula el centro de la red como criterio de comparación, y nodosQueSuperanLatencia devuelve los nodos de acceso cuya latencia a su data center supera un máximo dado.

ServicioConsultas reúne las cinco consultas propias del sistema, que se describen en la sección 6. CargadorRed lee la red desde dos archivos CSV, uno de nodos y otro de enlaces, e informa el archivo y el número de línea cuando encuentra un dato inválido. La clase Main carga la red de ejemplo y muestra el resultado de cada servicio.

3. Representación de los grafos

Se eligió representar los grafos con listas de adyacencia y no con una matriz. Una red de telecomunicaciones es un grafo poco denso: cada equipo se conecta con unos pocos vecinos, y en la red de ejemplo hay 42 enlaces para 24 nodos, cuando una red con un enlace entre cada par tendría 276. Con listas el espacio es $O(V + E)$, contra $O(V^2)$ de una matriz, y recorrer los vecinos de un vértice cuesta lo que su grado y no V. Esto favorece a los recorridos y a Dijkstra, que en cada paso solo miran los vecinos del vértice actual. Floyd y Warshall igualmente necesitan una matriz de $V \times V$, pero la arman internamente a partir de las listas.

La representación tiene un costo oculto que condiciona el análisis de todo el sistema. La interfaz del proyecto base no busca los vértices por clave sino mediante un Comparable: buscarVertice recibe un criterio y recorre los vértices hasta encontrar uno que lo cumpla, y adyacencias, existeArista y agregarArista lo usan internamente. Por eso, aunque el LinkedHashMap permitiría encontrar un vértice en O(1), cada una de esas operaciones cuesta

O(V). La consecuencia es que Dijkstra, que pide los adyacentes de cada vértice una vez, queda en $O(V^2 + E \log V)$ en lugar de $O(E \log V)$, y armar un grafo con E aristas cuesta $O(V + E \cdot V)$. Se evaluó cambiar buscarVertice para que usara el mapa directamente, pero eso implicaba modificar el comportamiento definido por el TDA, y se decidió mantenerlo. Este costo se tiene en cuenta en cada análisis de la sección 5.

Además de los grafos, los algoritmos usan estructuras auxiliares cuando las necesitan: mapas de costos y predecesores en Dijkstra, conjuntos de visitados en los recorridos, matrices de V × V en Floyd y Warshall y un mapa de componentes en Kruskal. RedTelecom usa dos LinkedHashMap, que conservan el orden en que se cargaron los datos. Ese orden se usa como criterio de desempate en varias operaciones, para que los resultados no dependan de detalles internos de los mapas.

4. Principales decisiones de diseño

Una de las principales decisiones fue separar los datos de la red de los grafos que se arman con ellos. Los grafos solo conocen identificadores, y RedTelecom es el único lugar donde viven los atributos de nodos y enlaces. De esta forma la biblioteca se usa tal cual, sin que sus clases sepan qué es un enlace de fibra o un punto de acceso, y cada dato aparece como peso únicamente en el grafo que lo necesita.

Otra decisión fue usar tres grafos en lugar de uno. Un único grafo dirigido con todos los atributos en las aristas alcanzaba para guardar la información, pero cada algoritmo necesita algo distinto: Dijkstra y Floyd necesitan arcos con la latencia de cada sentido, Kruskal necesita un grafo no dirigido con el costo como peso, y los puntos de articulación están definidos sobre grafos no dirigidos. Armar el grafo adecuado para cada pregunta permite usar los algoritmos de la biblioteca sin adaptarlos.

Para representar las fallas se evaluaron dos alternativas: marcar nodos y enlaces como no disponibles mediante un atributo, o armar el grafo sin ellos. En el primer modelo Nodo y Enlace tenían un atributo disponible, pero se descartó. Con un atributo, simular una falla obliga a modificar la red y a acordarse de restaurarla después, y dos simulaciones seguidas pueden interferir entre sí. Pasando los elementos caídos como parámetro, cada simulación es independiente, la red nunca cambia y se pueden evaluar muchos escenarios seguidos, que es justamente lo que necesita el ranking de nodos críticos.

La restricción de ancho de banda mínimo se resolvió filtrando los enlaces y no modificando Dijkstra. Dijkstra minimiza la suma de los pesos y no tiene forma de respetar una condición sobre cada arista del camino. En cambio, si el grafo se arma solo con los enlaces de al menos ese ancho de banda, cualquier camino que encuentre cumple la restricción, y sigue siendo el de menor latencia entre los que la cumplen.

En la distribución de contenido hubo que decidir en qué sentido se mide la latencia. El contenido viaja desde el data center hacia el usuario, así que la latencia que importa es la del data center al nodo, que en la matriz de Floyd es la fila del data center. La excentricidad del teórico, en cambio, se define con la columna, es decir, con las distancias de los demás vértices

hacia el vértice. Como la latencia es asimétrica, las dos medidas pueden dar resultados distintos. Para calcular el centro de la red con el criterio de la CDN sin modificar la biblioteca, centroParaDC aplica obtenerCentroGrafo sobre el grafo de ruteo invertido, en el que cada arco va en sentido contrario: la columna del grafo invertido es la fila del original.

Para recomendar dónde instalar un data center nuevo se eligió un criterio ponderado por usuarios. El centro de la red minimiza la peor latencia desde un nodo, pero no tiene en cuenta que ya existen otros data centers ni que los nodos atienden cantidades muy distintas de usuarios. Por eso el beneficio de un candidato se define como la suma, sobre los nodos de acceso, de los usuarios de cada nodo multiplicados por la reducción de latencia que obtendrían si ese candidato pasara a ser data center. Un nodo que ya está bien atendido no suma, y un nodo con muchos usuarios lejos de todo pesa más. Los nodos que hoy no llegan a ningún data center no se cuentan, porque su mejora sería infinita y dominaría la comparación. En los datos de ejemplo los dos criterios no coinciden: el centro de la red es RR1, mientras que el candidato con mayor beneficio es AP5, que es donde se concentran los usuarios de la zona rural.

Para los enlaces críticos, la biblioteca no tiene un algoritmo de puentes pero sí uno de puntos de articulación, y se aprovechó la relación entre ambos conceptos. Si se agrega un vértice m en el medio de cada enlace a-b, el enlace es un puente si y solo si m es punto de articulación: m solo toca a a y a b, así que sacarlo separa la red únicamente si no había otro camino entre ellos. No alcanza con mirar si los dos extremos son críticos; en un triángulo A-B-C con un nodo colgando de A y otro de B, tanto A como B son puntos de articulación y sin embargo el enlace A-B no es puente. Como vértice del medio se usó el propio objeto Enlace, en un grafo cuyos vértices son de tipo Object. Un identificador nunca es igual a un enlace, así que no hace falta inventar nombres que podrían coincidir con los de la red.

Para los nodos incomunicados se hace una búsqueda en amplitud desde los data centers que siguen funcionando, y los nodos que no se alcanzan quedan incomunicados. La búsqueda en amplitud de la interfaz no dirigida recorre el grafo completo y no recibe un origen, por lo que se usó la de la interfaz dirigida, que recibe cualquier IGraph y un vértice de partida. Solo se arranca una búsqueda desde un data center que todavía no fue alcanzado, de modo que cada componente se recorre una sola vez.

Al medir el impacto de una falla se decidió contar únicamente a quienes pierden el servicio por esa falla. Para eso se calculan los incomunicados sin fallas y con la falla, y se toma la diferencia: los nodos que ya estaban incomunicados antes no se cuentan, y si el nodo que cae ya estaba incomunicado, sus propios usuarios tampoco, porque ya no tenían servicio. En un primer diseño el resultado también informaba qué nodos cambiaban de data center, pero se quitó porque hacía que resiliencia dependiera de la asignación de la CDN. Esa información se puede obtener igual llamando a asignarCDN sin el data center caído.

Para el cableado de oficinas nuevas se descartó tomar las oficinas de la red y los enlaces que hay entre ellas, porque conectar solo algunos nodos pasando por otros es un problema distinto, el del árbol de Steiner, y no un árbol abarcador. En su lugar, para planificar se arma una RedTelecom con las oficinas a conectar y las conexiones posibles con su costo, y se le

pide el cableado mínimo, que es lo que describe la letra. Se eligió Kruskal y no Prim porque, si alguna oficina no tiene conexiones posibles, Kruskal devuelve el mejor cableado para cada grupo que sí se puede unir, mientras que Prim solo devolvería el árbol del grupo donde empezó.

En cuanto a las clases de resultado (RutaResultado, TablaRuteo, ImpactoFalla, AsignacionCDN, RecomendacionDC, ResultadoCableado y ResultadoDiametro), reúnen en un solo objeto lo que devuelve cada servicio. Las que reciben listas, conjuntos o mapas los copian, de modo que si quien llamó modifica después su colección original, el resultado no cambia. La excepción es TablaRuteo, que guarda el resultado de Floyd ya calculado.

5. Análisis de complejidad

En lo que sigue, V es la cantidad de nodos de la red, E la cantidad de enlaces y D la cantidad de data centers. Como se explicó en la sección 3, la búsqueda de vértices de la biblioteca cuesta O(V), y esto cambia el costo real de varias operaciones respecto del que se esperaría con listas de adyacencia. Para cada operación se indica el costo con la implementación actual y de dónde viene.

Armar cualquiera de los tres grafos cuesta $O(V + E \cdot V)$. Agregar los V vértices es O(V), y cada una de las E aristas requiere verificar que no exista ya, lo que busca el vértice de origen en O(V) y recorre su lista. Como el grafo se arma de nuevo en cada operación, este costo aparece en todas, y en las de ruteo y resiliencia es del mismo orden que el propio algoritmo.

rutaMenorLatencia cuesta $O(V^2 + E \cdot V)$. Dijkstra saca cada vértice de la cola una vez y pide sus adyacentes, lo que suma $O(V^2)$ por la búsqueda de cada vértice, y cada mejora de un costo encola una entrada en O(log V), lo que en el peor caso suma O(E log V). A eso se le agrega armar el grafo, cuyo término $O(E \cdot V)$ domina al O(E log V). El cuello de botella se calcula recorriendo la ruta y buscando cada enlace en el mapa de RedTelecom, en O(1) por enlace. rutaConAnchoBandaMinimo y rutaAlternativa tienen el mismo costo, ya que el filtrado se hace mientras se arma el grafo y no agrega trabajo.

tablaDeRuteo cuesta $O(V^3)$. Floyd arma la matriz pidiendo los adyacentes de cada vértice, en $O(V^2 + E)$, y después ejecuta los tres bucles anidados sobre k, i y j. Una vez construida la tabla, consultar la latencia entre dos nodos cuesta O(1) y reconstruir un camino cuesta lo que su largo. hayCamino usa Warshall, también $O(V^3)$, y diametro le agrega a Floyd un recorrido de la matriz en $O(V^2)$, así que ambas operaciones quedan en $O(V^3)$.

cableadoMinimo cuesta $O(V^2 + E \cdot V)$. Kruskal ordena las aristas en O(E log E) y, por cada una de las $V - 1$ aristas que agrega, reetiqueta los vértices de una componente en O(V), lo que suma $O(V^2)$. Armar el grafo agrega $O(E \cdot V)$, que domina al ordenamiento porque log E crece mucho más lento que V. Pasar las aristas del árbol a enlaces de la red cuesta O(1) cada una gracias a la clave por par de RedTelecom.

nodosCriticos cuesta $O(V^2 + E \cdot V)$. La búsqueda en profundidad visita cada vértice una vez y recorre cada arista desde sus dos extremos, pero como pide los adyacentes de cada vértice queda en $O(V^2 + E)$, y armar el grafo domina. enlacesCriticos trabaja sobre el grafo

subdividido, que tiene V + E vértices y 2E aristas, así que su costo es $O((V + E)^2)$. Es más cara que nodosCriticos, a cambio de no tener que implementar un algoritmo de puentes aparte.

incomunicados cuesta $O(V^2 + E \cdot V)$: armar el grafo sin los elementos caídos y una búsqueda en amplitud que, sumando todos los data centers, visita cada componente una sola vez. impactoFallaNodo e impactoFallaEnlace calculan los incomunicados dos veces, sin la falla y con ella, y tienen el mismo orden. rankingCriticosSimple repite impactoFallaNodo para cada uno de los V nodos, por lo que cuesta $O(V^3 + E \cdot V^2)$, más $O(V \log V)$ para ordenar el resultado.

asignarCDN cuesta $O(V^3)$ por Floyd, más $O(V \cdot D)$ para elegir, para cada nodo, el data center con menor latencia mirando la fila de cada uno. centroParaDC y nodosQueSuperanLatencia también quedan en $O(V^3)$, la segunda con un $O(V \log V)$ adicional para ordenar. recomendarDCSimple llama a asignarCDN una vez por cada candidato, y como cada llamada recalcula Floyd, cuesta $O(V^4)$. Es la operación más cara del sistema y la que se optimizó en el Desafío 3, donde se analiza en detalle.

Cargar la red desde los archivos cuesta $O(V + E)$: cada línea se valida y se agrega a los mapas de RedTelecom en $O(1)$.

En cuanto al espacio, RedTelecom ocupa $O(V + E)$, y cada grafo que se arma también, ya que las listas de adyacencia guardan una entrada por arco: dos por enlace, tanto en el grafo de ruteo como en los no dirigidos. Las operaciones que usan Floyd o Warshall necesitan además matrices de $V \times V$, $O(V^2)$, que son el mayor costo espacial del sistema. La tabla 1 resume los costos temporales.

Tabla 1. Costo temporal de las operaciones principales del sistema.

Operación	Algoritmo	Costo
Armar un grafo (ruteo, cableado o resiliencia)	Construcción	$O(V + E \cdot V)$
rutaMenorLatencia, rutaConAnchoBandaMinimo, rutaAlternativa	Dijkstra	$O(V^2 + E \cdot V)$
tablaDeRuteo, diametro	Floyd	$O(V^3)$
hayCamino	Warshall	$O(V^3)$
cableadoMinimo	Kruskal	$O(V^2 + E \cdot V)$
nodosCriticos	Puntos de articulación	$O(V^2 + E \cdot V)$
enlacesCriticos	Articulación sobre el grafo subdividido	$O((V + E)^2)$
incomunicados, impactoFallaNodo, impactoFallaEnlace	BEA desde los data centers	$O(V^2 + E \cdot V)$
rankingCriticosSimple	Una simulación por nodo	$O(V^3 + E \cdot V^2)$
asignarCDN, centroParaDC, nodosQueSuperanLatencia	Floyd	$O(V^3)$
recomendarDCSimple	Floyd por cada candidato	$O(V^4)$
recomendarDC (Desafío 3)	Floyd una sola vez	$O(V^3)$

6. Decisiones tomadas en el Desafío 2

El Desafío 2 pedía transformar el escenario del proveedor en un modelo basado en grafos. La primera decisión fue qué representan los vértices y las aristas. Los vértices son los equipos de la red y las aristas son los enlaces físicos entre ellos. Los usuarios no se modelaron como vértices sino como un atributo de los nodos de acceso: representar cada dispositivo agrandaría el grafo sin aportar información a ninguna de las preguntas planteadas, porque lo que importa de un usuario es a qué nodo está conectado. Los data centers, en cambio, sí son nodos del grafo, así que pueden fallar como cualquier otro equipo y participan de las rutas.

Para que las firmas fueran claras antes de implementar los servicios, primero se definieron el dominio, RedTelecom y los grafos, y después un esqueleto de cada servicio con sus métodos documentados pero sin implementar. Con ese contrato fijo, ruteo y CDN por un lado y cableado y resiliencia por otro pudieron desarrollarse en paralelo, cada uno sobre sus propios archivos.

Los datos de ejemplo se armaron para que cada servicio tenga algo que mostrar, y lo que tiene la red a propósito quedó escrito como comentario en los propios archivos. Son 24 nodos y 42 enlaces, con tres data centers y 11220 usuarios. El núcleo es redundante (un anillo de cuatro routers con una diagonal), así que la caída de cualquiera de sus routers no deja a nadie sin servicio. Una zona rural cuelga del router R5 por un único radioenlace, lo que hace de R5, RR1 y AP5 puntos de articulación y de cuatro enlaces puentes, y deja a 6800 usuarios lejos de todos los data centers. Hay enlaces con distinta latencia en cada sentido, un radioenlace angosto que es el camino más rápido entre dos switches y dos caminos de igual latencia entre R2 y R4. La prueba DatosDeEjemploTest verifica que los datos tengan efectivamente cada una de estas propiedades.

La letra exige al menos cinco operaciones o consultas que requieran los grafos y los algoritmos desarrollados y que no consistan en mostrar información almacenada. Se eligieron las cinco que siguen, todas en ServicioConsultas, y para cada una se indica para qué sirve, cómo se resuelve y qué devuelve con la red de ejemplo.

La primera es rutaConAnchoBandaMinimo(origen, destino, anchoMinimo), que devuelve la ruta de menor latencia que garantiza un ancho de banda mínimo, junto con su cuello de botella. Es la pregunta que se hace un proveedor antes de mandar video o un respaldo grande: el camino más rápido no sirve si pasa por un enlace angosto. Se resuelve filtrando los enlaces y aplicando Dijkstra. Entre SW2 y SW3 la ruta más rápida es el radioenlace directo, con 3 ms pero solo 50 Mbps; pidiendo al menos 1000 Mbps la ruta pasa a ser SW2, R3, R4, SW3, con 13 ms. La letra menciona el ancho de banda mínimo como algo que se valora dentro del ruteo; la consulta agrega el cuello de botella de la ruta, que el requisito mínimo no pide.

La segunda es queOcurreSiCae(nodo), que devuelve qué nodos se quedan sin data center si cae un equipo y cuántos usuarios se ven afectados. Se resuelve con dos búsquedas en amplitud desde los data centers vivos, con y sin la falla. Si cae R5 quedan incomunicados RR1, AP5, AP6 y OF5, con 6800 usuarios afectados. La resiliencia ya pide determinar qué nodos

quedan incomunicados; lo que agrega esta consulta es la medida de usuarios afectados, que es la que permite comparar una falla con otra.

La tercera es rankingNodosCriticos(), que ordena los equipos según cuántos usuarios se quedarían sin servicio si fallaran, para saber dónde conviene invertir primero en redundancia. Simula la caída de cada nodo y ordena los resultados. En la red de ejemplo encabezan R5 y RR1 con 6800 usuarios, seguidos por AP5 con 6300 y AP6 con 2800. Es distinta de los puntos de articulación: AP2 no es punto de articulación y aun así aparece con 1200 usuarios, porque su caída deja sin servicio a sus propios usuarios, mientras que los routers del núcleo no aparecen porque el núcleo es redundante.

La cuarta es diametro(), que devuelve el par de nodos con la mayor latencia mínima, es decir, el peor caso de la red. Se calcula recorriendo la matriz de Floyd fuera de la diagonal. Como la latencia es asimétrica, el diámetro es un par ordenado: en la red de ejemplo es de AP6 hacia AP1, con 122 ms. Si algún par de nodos no tiene camino, el diámetro es infinito y se informa el primer par en esa situación.

La quinta es nodosQueSuperanLatencia(maximo), que devuelve los nodos de acceso cuya latencia a su data center supera un máximo, ordenados de mayor a menor. Responde qué usuarios no reciben el nivel de servicio comprometido. Se apoya en la asignación de la CDN, que usa Floyd y la fila de cada data center. Con un máximo de 30 ms devuelve AP6, AP5 y OF5, toda la zona rural, lo que coincide con la recomendación de instalar ahí un data center nuevo.

Para demostrar el funcionamiento de las decisiones se escribieron 260 pruebas orientadas a la solución, separadas de las de la biblioteca. Cubren las validaciones del dominio, los errores del cargador con su número de línea, los tres grafos de RedTelecom (que los arcos tengan la latencia de su sentido, que el filtro respete el borde del ancho mínimo, que las fallas no modifiquen la red) y cada servicio con ejemplos calculados a mano. También incluyen comparaciones cruzadas: la ruta de Dijkstra contra la tabla de Floyd, el costo del cableado contra el de Prim en 40 redes al azar, los puentes contra su definición en 50 redes al azar y las consultas contra el servicio que las resuelve. Como en la biblioteca, se introdujeron errores a propósito en copias del código para verificar que las pruebas los detectaran. En total el proyecto tiene 462 casos de prueba, que pasan con mvn test.

7. Optimización realizada en el Desafío 3

El Desafío 3 pide identificar una operación cuya eficiencia pueda mejorarse, medir o analizar su costo, proponer una alternativa, implementarla y comparar ambas soluciones. Del análisis de la sección 5 surgieron dos candidatas, y se decidió implementar una sola de ellas.

La operación elegida es la recomendación de un data center nuevo. La versión original, recomendarDCSimple, prueba cada nodo que no es data center como candidato: lo agrega al conjunto de data centers, llama a asignarCDN y compara las latencias resultantes con las actuales. Es la forma natural de escribirla reutilizando una operación que ya existía, pero

asignarCDN arma el grafo de ruteo y corre Floyd en cada llamada, así que la recomendación hace V corridas de Floyd y cuesta $O(V^4)$.

Lo que se repite sin necesidad es Floyd. Agregar un data center no cambia ni los enlaces ni las latencias de la red, por lo que la matriz de distancias es exactamente la misma en todas las pruebas; lo único que cambia de una prueba a otra es qué nodos son data center. La alternativa, recomendarDC, calcula Floyd una sola vez y guarda la latencia actual de cada nodo de acceso a su data center, que es el mínimo entre las filas de los data centers existentes. Si un candidato c pasara a ser data center, la nueva latencia de un nodo u sería el mínimo entre su latencia actual y la distancia de c hacia u, así que el beneficio de c se puede calcular directamente a partir de la matriz, mirando la fila de c: la suma de los usuarios de u multiplicados por la diferencia entre la latencia actual y la distancia de c a u, solo cuando esa diferencia es positiva.

Con esto el costo pasa de $O(V^4)$ a $O(V^3)$. Floyd se ejecuta una vez, en $O(V^3)$, las latencias actuales cuestan $O(V^2)$ (para cada nodo de acceso se vuelve a armar la lista de data centers y se la recorre) y evaluar los candidatos cuesta $O(V^2)$, ya que para cada uno de los V candidatos se recorren los V nodos. Es un caso de reutilización de resultados: no cambia la representación ni el algoritmo de caminos mínimos, sino la cantidad de veces que se lo ejecuta.

La versión optimizada respeta todas las reglas de la original: solo cuentan los nodos de acceso, los nodos que hoy no llegan a ningún data center no suman, los usuarios beneficiados son los de los nodos cuya latencia mejora estrictamente, en caso de empate gana el candidato que aparece primero en la red y, si no hay candidatos, el resultado es nulo. Como las dos versiones calculan con los mismos valores de la misma matriz y suman en el mismo orden, devuelven exactamente el mismo candidato, el mismo beneficio y la misma cantidad de usuarios. La equivalencia se verificó con 14 pruebas específicas: los mismos casos calculados a mano que la versión original, la red de ejemplo, 40 redes del generador entre árboles y mallas y 40 redes armadas al azar que pueden no ser conexas o no tener data centers. Las dos versiones quedaron en el código, y el Main muestra ambas.

La otra candidata fue el ranking de nodos críticos. rankingCriticosSimple simula la caída de cada nodo y cuesta $O(V^3 + E \cdot V^2)$, pero si cae un nodo que no es punto de articulación ni data center, el resto de la red sigue conectada y solo pierden el servicio sus propios usuarios. Una versión optimizada calcularía una vez los puntos de articulación y el estado sin fallas, y simularía solo esos nodos y los data centers, con un costo de $O(k \cdot (V^2 + E \cdot V))$, siendo k la cantidad de nodos simulados. La idea está respaldada por una prueba que lo verifica en 30 redes al azar, pero no se implementó. Su mejora depende de la forma de la red: en una malla hay pocos puntos de articulación y la ganancia sería grande, pero en un árbol casi todos los nodos internos lo son, k es del orden de V y la ganancia se reduce a no recalcular el estado sin fallas. La optimización de la CDN, en cambio, ahorra un factor V con cualquier forma de red, y por eso se eligió. Una mejora todavía mayor para el ranking sería calcular en un único recorrido en profundidad, para cada punto de articulación, cuántos usuarios y data centers quedan de cada lado, lo que bajaría el costo a $O(V + E)$ si la búsqueda de un vértice fuera

O(1). Con la búsqueda lineal de esta biblioteca, y armando el grafo una sola vez, quedaría en $O(V^2 + E \cdot V)$.

8. Resultados de los experimentos realizados

Para medir las dos versiones se escribió GeneradorRedes, que arma redes al azar a partir de una semilla, una cantidad de nodos, una cantidad de enlaces extra y una cantidad de data centers. Para que la red sea siempre conexa, primero arma un árbol al azar uniendo cada nodo con uno de los anteriores y después agrega los enlaces extra entre pares que todavía no están unidos: sin extra la red es un árbol, y con más se vuelve una malla. Las latencias van de 1 a 20 ms, y alrededor de un 30% de los enlaces tienen distinta latencia en cada sentido. Con la misma semilla el generador devuelve siempre la misma red.

El experimento está en BenchmarkCDN. Usa redes con 2V enlaces extra, es decir, alrededor de 3V enlaces, que corresponde a una malla poco densa como la de una red real, y un data center cada 25 nodos, con la semilla igual a V para que cada tamaño sea reproducible. Antes de medir, verifica que las dos versiones den el mismo resultado. Después corre una etapa de calentamiento general y, para cada tamaño, tres ejecuciones sin medir de cada versión y siete ejecuciones medidas con System.nanoTime(), alternando las dos versiones. Se toma la mediana, que no se ve afectada por una ejecución aislada más lenta. El calentamiento general se agregó después de una primera prueba en la que la red de 25 nodos tardaba más que la de 50, porque la máquina virtual de Java todavía estaba compilando el código. Los resultados quedan guardados en un archivo CSV dentro de la carpeta resultados del proyecto y se resumen en la tabla 2.

Tabla 2. Tiempo de la recomendación de un data center nuevo, mediana de 7 ejecuciones, en milisegundos.

Nodos	Enlaces	Data centers	recomendarDCSimple	recomendarDC	Aceleración
50	149	2	10,58	0,59	17,9
100	299	4	108,31	1,38	78,3
150	449	6	366,95	2,58	142,1
200	599	8	979,31	5,40	181,5
300	899	12	4534,39	16,49	275,0
400	1199	16	13442,04	37,00	363,3

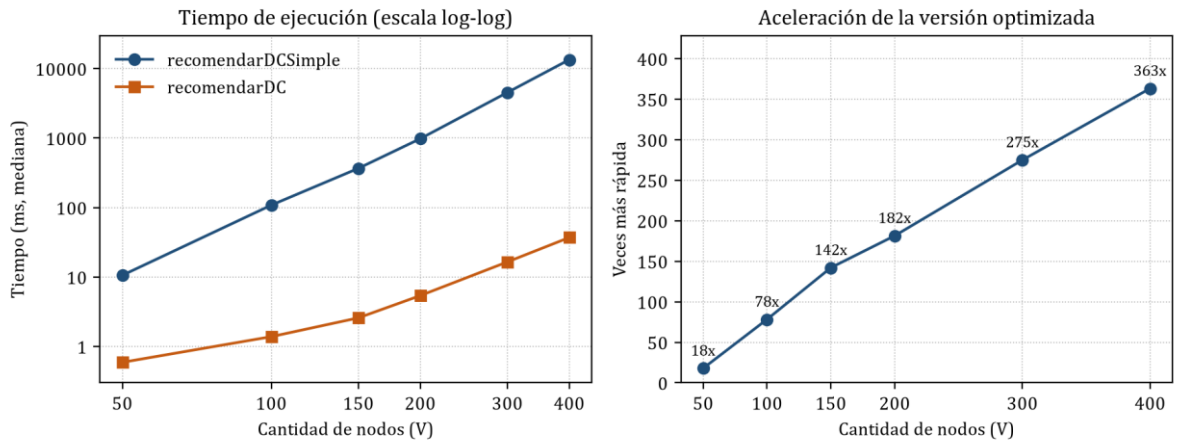


Figura 1. Tiempo de ambas versiones en escala logarítmica y aceleración de la versión optimizada según la cantidad de nodos.

La diferencia crece con el tamaño de la red. Con 50 nodos la versión optimizada es unas 18 veces más rápida, y con 400 nodos la versión original tarda más de 13 segundos contra 37 milisegundos de la optimizada, unas 363 veces menos. Para una red de varios cientos de equipos, esto es la diferencia entre esperar varios segundos por cada recomendación y obtenerla al instante.

La forma de las curvas confirma el análisis teórico. Al duplicar la cantidad de nodos de 200 a 400, la versión original tardó 13,7 veces más y la optimizada 6,9 veces más, cuando la teoría predice 16 y 8 para $O(V^4)$ y $O(V^3)$. En escala logarítmica la pendiente entre esos dos tamaños es de 3,8 para la original y de 2,8 para la optimizada, y entre 300 y 400 nodos da prácticamente lo mismo. Las pendientes quedan algo por debajo de 4 y 3 porque, con estos tamaños, todavía pesan los términos de menor orden, como armar el grafo en $O(E \cdot V)$ o recorrer los candidatos en $O(V^2)$, y en los tamaños más chicos el tiempo de la versión optimizada está por debajo de un par de milisegundos y es más sensible a variaciones de la máquina. La aceleración crece de forma aproximadamente proporcional a V, que es justamente el factor que se ahorra al no recalcular Floyd para cada candidato.

Las mediciones se hicieron sobre mallas poco densas. Como el costo de Floyd no depende de la cantidad de enlaces, se espera un comportamiento parecido en árboles o en redes más densas, aunque eso no se midió por separado.

9. Declaración de uso de herramientas de IA

En este proyecto se utilizó Claude Code, un asistente de programación con IA generativa, principalmente para las pruebas, los benchmarks y la forma en que los servicios devuelven sus resultados. Se detalla a continuación, manteniendo el criterio de los informes anteriores: qué se le pidió, qué surgió de eso y qué decidió el equipo. En todos los casos el código y el texto incorporados fueron revisados por el equipo.

El uso principal fueron las pruebas. La mayoría de los casos de prueba se redactaron con la herramienta, a partir de la interfaz y del comportamiento esperado de cada clase, y también

se la usó para revisar que no quedaran casos sin cubrir, empezando por los casos borde que pide la letra (grafo vacío, un único vértice, grafos no conexos, vértices inexistentes, empates). Para no depender de que las pruebas estuvieran bien escritas, se verificó que detectaran errores reales introduciendo errores a propósito en copias aisladas del código y comprobando que efectivamente fallaran.

El segundo uso fueron los benchmarks. La herramienta generó GeneradorRedes y BenchmarkCDN a partir de lo que el equipo definió: comparar recomendarDCSimple con recomendarDC en redes de distintos tamaños, verificar antes de medir que las dos versiones den el mismo resultado, calentar la máquina virtual y quedarse con la mediana de las ejecuciones. El equipo corrió el experimento y comparó los resultados con el análisis teórico.

El tercer uso fue la forma en que se reciben los resultados de los servicios. Las clases del paquete resultado y sus métodos toString se redactaron con la herramienta, para que lo que muestra el Main y lo que se compara en las pruebas sea fácil de leer. Qué datos debía devolver cada operación lo definió el equipo.

Además, la herramienta se usó para organizar las tareas del proyecto, no para tomar decisiones. A partir de la consigna y del código base se armó un plan por desafíos y se repartieron las tareas por archivos, para que cada integrante pudiera trabajar en una parte sin interferir con las demás. Las decisiones de diseño y modelado, incluida la elección de la optimización del Desafío 3, fueron discutidas y tomadas por el equipo. Por último, la herramienta redactó un borrador de este informe a partir de la bitácora que el equipo llevó durante el proyecto, así como se le solicitó escribir comentarios en el código realizado para la mejor comprensión de todos los integrantes, donde se detallaban las decisiones tomadas y los resultados obtenidos.

10. Conclusión

El desarrollo de este proyecto mostró que, al trabajar con grafos, decidir sobre qué grafo aplicar cada algoritmo pesa tanto como implementarlo. La misma red de telecomunicaciones terminó representada por tres grafos distintos, y la elección entre dirigido y no dirigido o entre un peso y otro salió en cada caso de la pregunta que había que responder. Resolver eso antes de escribir los servicios permitió usar los algoritmos de la biblioteca sin modificarlos, incluso en casos que la biblioteca no cubría directamente, como los puentes o el centro medido desde el data center.

También quedó en evidencia que el costo real de una operación depende de detalles de la representación que no se ven a simple vista. La búsqueda lineal de vértices que impone la interfaz hace que operaciones que en teoría son casi lineales en la cantidad de enlaces terminen siendo cuadráticas, y ese efecto aparece en todo el análisis de complejidad. Haberlo identificado temprano permitió explicar los costos con precisión y no atribuirle a las listas de adyacencia propiedades que, con esta interfaz, no se cumplen del todo.

La optimización del Desafío 3 mostró que no siempre hace falta cambiar de estructura o de algoritmo para mejorar una operación. La recomendación de un data center usaba el mismo

algoritmo antes y después; lo que cambió fue dejar de recalcular un resultado que no cambiaba, y eso alcanzó para bajar un orden de complejidad y llegar a una diferencia de más de 300 veces con 400 nodos. Comparar las dos candidatas también sirvió para entender que una optimización vale más cuando su mejora no depende de la forma de los datos.

Más allá de lo técnico, el proyecto obligó a trabajar en paralelo sobre una misma base de código. Acordar primero las firmas y dividir el trabajo por archivos evitó que el trabajo de una parte del equipo bloqueara al de la otra, y llevar una bitácora de las decisiones, incluyendo las que cambiaron en el camino, fue lo que permitió reconstruir y justificar cada una de ellas al escribir este informe.